Major Project

CloudOps Enterprise Platform

Cloud Infrastructure, Automation & Deployment Ecosystem

Step-by-step implementation work document

---

1. Project Objective

Build and demonstrate an integrated cloud and DevOps platform using Terraform-managed AWS infrastructure, Dockerized microservices, and an event-driven serverless processing pipeline. The project focuses on repeatable infrastructure provisioning, authenticated service APIs, reliable booking-event publication, asynchronous processing, secure storage, and CI validation.

Tools Used

Terraform, AWS CLI, Amazon VPC, IAM, SQS, Lambda, S3, DynamoDB, CloudWatch, Docker, Docker Compose, Nginx, Node.js, Express, PostgreSQL, Python, Git and GitHub Actions.

Tasks Implemented

- Provision an isolated AWS VPC foundation with public and private subnets.
- Run Auth, Booking and Notification APIs as separate local containers behind an Nginx gateway.
- Persist a booking and its event in one PostgreSQL transaction using the transactional outbox pattern.
- Publish events to SQS using a short-lived assumed IAM role.
- Process events with Lambda and store data in S3 and DynamoDB.
- Verify retry handling, dead-letter routing, infrastructure configuration and GitHub Actions CI.

2. Project Status

The status below reflects work that was actually implemented and tested. A repository folder or placeholder is not counted as a completed module.

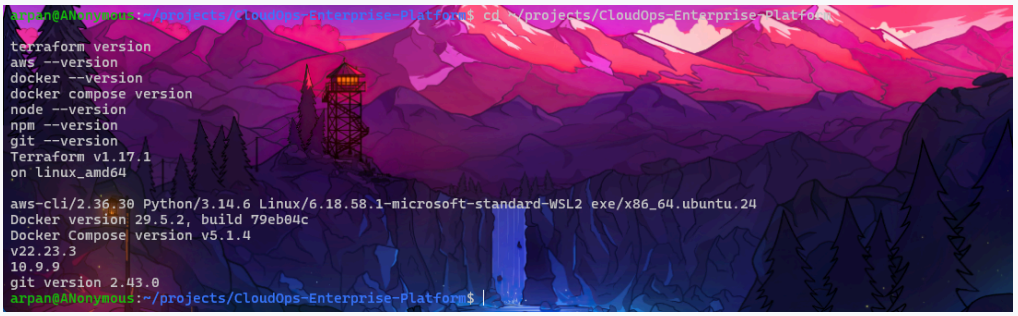
Workstream	Status	Implementation status
1. Infrastructure & virtualization	Partial	Terraform VPC foundation applied; application containers run locally. No compute workload deployed inside the CloudOps VPC.
2. Networking, storage & data	Partial	AWS network, S3 and DynamoDB implemented; PostgreSQL runs locally rather than in RDS.
3. Serverless & microservices	Implemented / tested	Three local APIs and an AWS SQS-Lambda-S3-DynamoDB event pipeline.
4. Security, IAM & monitoring	Partial	JWT, scoped IAM, encryption, S3 public-access block and CloudWatch Logs. No full dashboards/alarms.
5. Infrastructure as Code	Implemented	Terraform provisioning, validation, plan review and apply.
6. CI/CD	Partial	GitHub Actions builds and validates; automated deployment to AWS is not configured.
7. Big Data / AI	Not implemented	No Spark/EMR analytics or model training/inference was executed.
8. No-code integration	Not implemented	No no-code application was deployed or integrated.

3. Environment

Environment item	Recorded value
Development OS	Windows with WSL 2, Ubuntu 24.04 distribution
Docker	Docker Desktop WSL integration; Docker Engine/CLI 29.5.2
Docker Compose	5.1.4
Terraform	1.17.1
AWS CLI	2.36.30
Node.js / npm	Node.js 22.23.3 / npm 10.9.9
Git	2.43.0
Application images	node:22-alpine; nginx:alpine; PostgreSQL 16
AWS region	Asia Pacific (Mumbai), ap-south-1
Lambda runtime	Python 3.14

Versions are the versions recorded during implementation. Re-check them on the machine if a later environment capture shows different values.

SCREENSHOT SLOT 01 | ENVIRONMENT VERSIONS



Captured 11 November 2026. WSL 2 environment versions: Terraform, AWS CLI, Docker, Compose, Node.js, npm and Git.

4. Architecture Overview

The architecture has two separate execution environments. The local environment hosts the service APIs and PostgreSQL containers. AWS hosts the VPC foundation and event-processing resources. The outbox worker calls the AWS SQS API using temporary STS credentials.

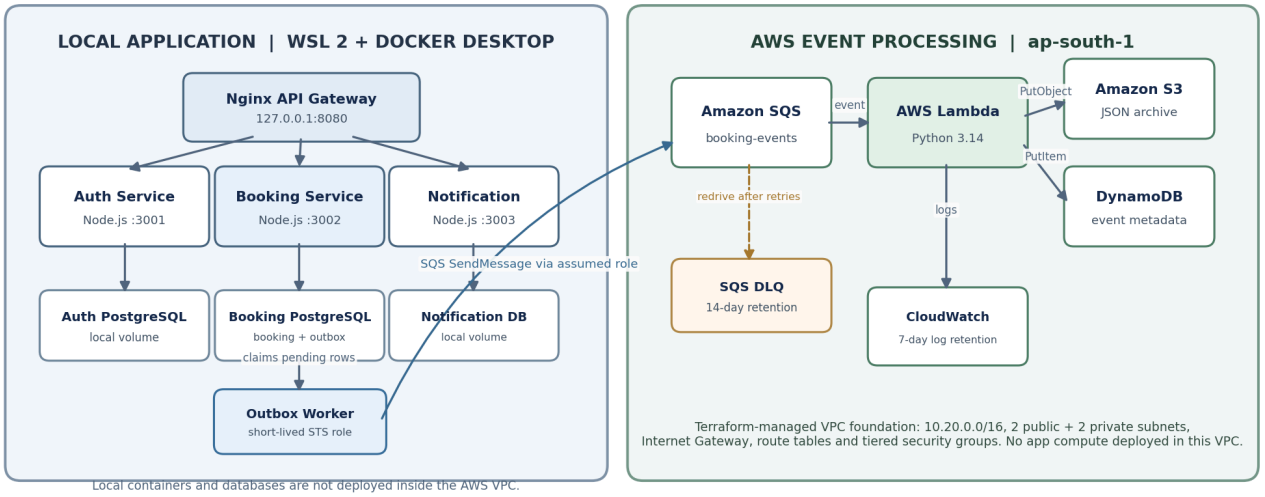


Figure 1. Implemented local application and AWS event-processing architecture.

Local Application Layer

Component	Role	Endpoint / storage
Nginx API Gateway	Routes API requests	127.0.0.1:8080
Auth Service	Registration, login, JWT	Docker network :3001
Booking Service	Booking CRUD + outbox	Docker network :3002
Notification Service	Notification API	Docker network :3003
PostgreSQL databases	Separate service persistence	Host ports 5433, 5434, 5435

AWS Layer

Terraform manages a VPC (10.20.0.0/16), two public and two private subnets, an Internet Gateway, route tables, and tiered security groups. The serverless event stack contains SQS, a DLQ, Lambda, S3, DynamoDB, IAM roles and a CloudWatch log group.

5. Step 1: Prepare the Project and Terraform Environment

The repository was maintained in WSL under the CloudOps Enterprise Platform project directory. Terraform configuration is stored in infrastructure/terraform. AWS credentials were configured for the AWS CLI and verified before applying infrastructure changes.

1.1 Open the repository and inspect tool versions

```
cd ~/projects/CloudOps-Enterprise-Platform

terraform version
aws --version
docker --version
docker compose version
node --version
npm --version
git --version
```

1.2 Verify the AWS identity

```
aws sts get-caller-identity --query Arn --output text
```

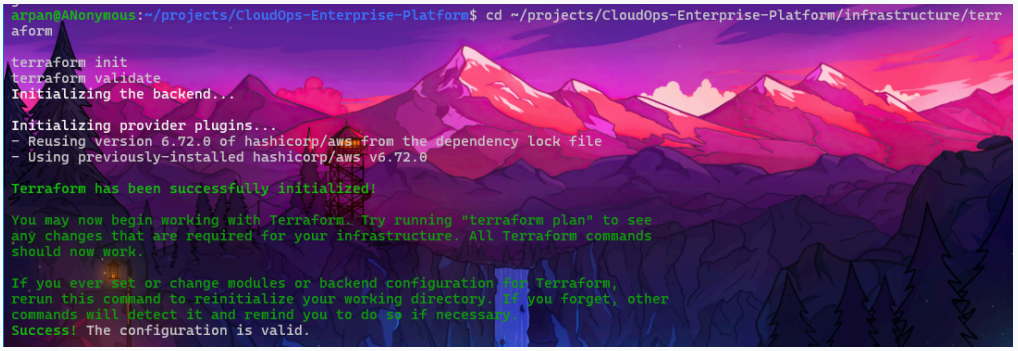
The command confirmed the AWS CLI principal used for Terraform. Do not include access keys, secret keys, private key files or session tokens in the report or screenshots.

1.3 Initialize and validate Terraform

```
cd ~/projects/CloudOps-Enterprise-Platform/infrastructure/terraform
terraform init
terraform fmt
terraform validate
```

Terraform init installs the required provider. Terraform fmt normalizes HCL formatting. Terraform validate checks configuration syntax and internal references; it does not replace a plan review.

SCREENSHOT SLOT 02 | TERRAFORM INITIALIZATION AND VALIDATION



Captured 11 November 2026. Terraform provider initialization completed and terraform validate reported a valid configuration.

6. Step 2: Provision the CloudOps VPC and Network

The AWS network foundation was defined in Terraform using reusable resource files. It was created separately from the existing Voyage VPC and did not modify the Voyage instance.

2.1 Network design

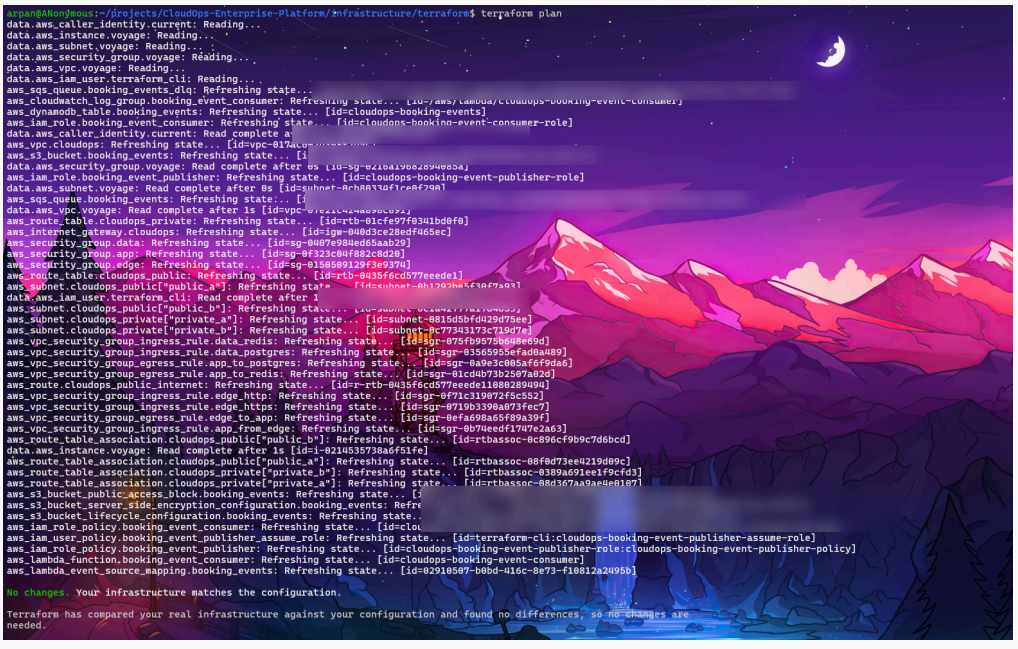
Resource	Configuration
CloudOps VPC	10.20.0.0/16
Public subnets	10.20.1.0/24 (ap-south-1a), 10.20.2.0/24 (ap-south-1b)
Private subnets	10.20.11.0/24 (ap-south-1a), 10.20.12.0/24 (ap-south-1b)
Internet Gateway	Attached to CloudOps VPC
Public route table	Default route to Internet Gateway
Private route table	Local VPC route only; no NAT Gateway
Security groups	Edge, application and data tiers

2.2 Review, plan and apply

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

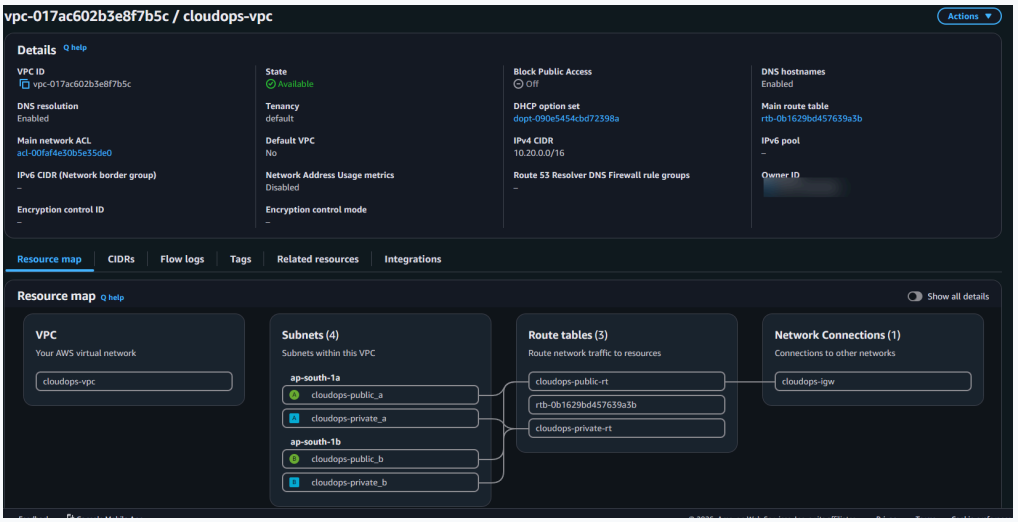
The plan was reviewed before applying. Terraform created the VPC, subnet, routing, Internet Gateway and security-group resources. A later terraform plan was also used to review the serverless components. Always inspect the add/change/destroy counts before applying.

SCREENSHOT SLOT 03 | TERRAFORM FINAL PLAN



Captured 11 November 2026. Terraform refreshed the managed resources and returned No changes. The infrastructure matched the configuration.

SCREENSHOT SLOT 04 | AWS VPC AND SUBNETS



Captured 11 November 2026. AWS Console resource map for the CloudOps VPC, showing four subnets across two Availability Zones, route tables and Internet Gateway.

7. Step 3: Create SQS, DLQ, DynamoDB and S3 Resources

The serverless data plane uses a queue for asynchronous booking events, a dead-letter queue for repeated failures, DynamoDB for event metadata and S3 for the raw JSON archive.

3.1 Queue configuration

Setting	Configured value
Main queue	cloudops-booking-events
Dead-letter queue	cloudops-booking-events-dlq
Visibility timeout	180 seconds
Redrive threshold	5 receives (updated after the initial DLQ test)
Main message retention	4 days
DLQ retention	14 days
Encryption	SQS-managed encryption

3.2 DynamoDB and S3

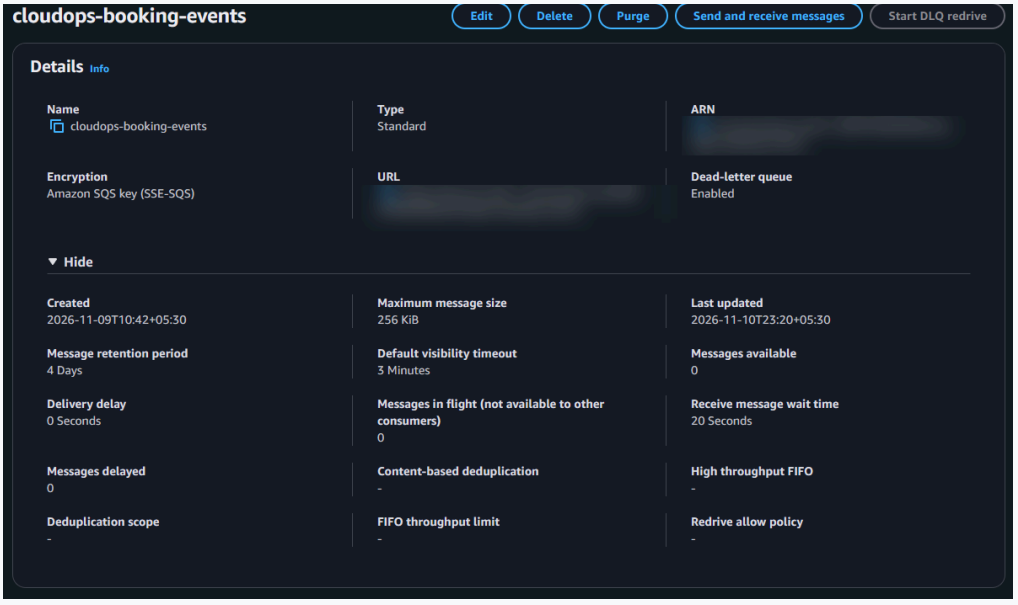
- DynamoDB table: cloudops-booking-events. Billing mode: PAY_PER_REQUEST. Partition key: event_id. TTL attribute: expires_at.
- S3 bucket: account- and region-specific cloudops-booking-events bucket. Public access is blocked and default server-side encryption is AES-256.
- Lifecycle rule expires objects under booking-events/ after 30 days.

3.3 Terraform resources

The resources are defined in serverless-storage-queues.tf, with supporting IAM, Lambda and output files. They were applied in the Terraform working directory after plan review.

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

SCREENSHOT SLOT 05 | SQS QUEUE AND DLQ SETTINGS



Captured 11 November 2026. Main booking-events SQS queue details, SQS-managed encryption, three-minute visibility timeout and enabled DLQ. The final redrive threshold of five is verified separately by AWS CLI.

8. Step 4: Package and Deploy the Lambda Consumer

The Python consumer reads SQS records, validates the event structure and identifiers, stores the raw event in S3, and writes metadata to DynamoDB. It uses a conditional write to avoid duplicate metadata records for the same event ID.

4.1 Lambda settings

Setting	Value
Function	cloudops-booking-event-consumer
Runtime	Python 3.14
Memory / timeout	128 MB / 20 seconds
Event source	Amazon SQS booking-events queue
Batch size / window	5 messages / 5 seconds
Partial batch failures	ReportBatchItemFailures enabled
Maximum event-source concurrency	2
CloudWatch log retention	7 days

4.2 Build deployment package

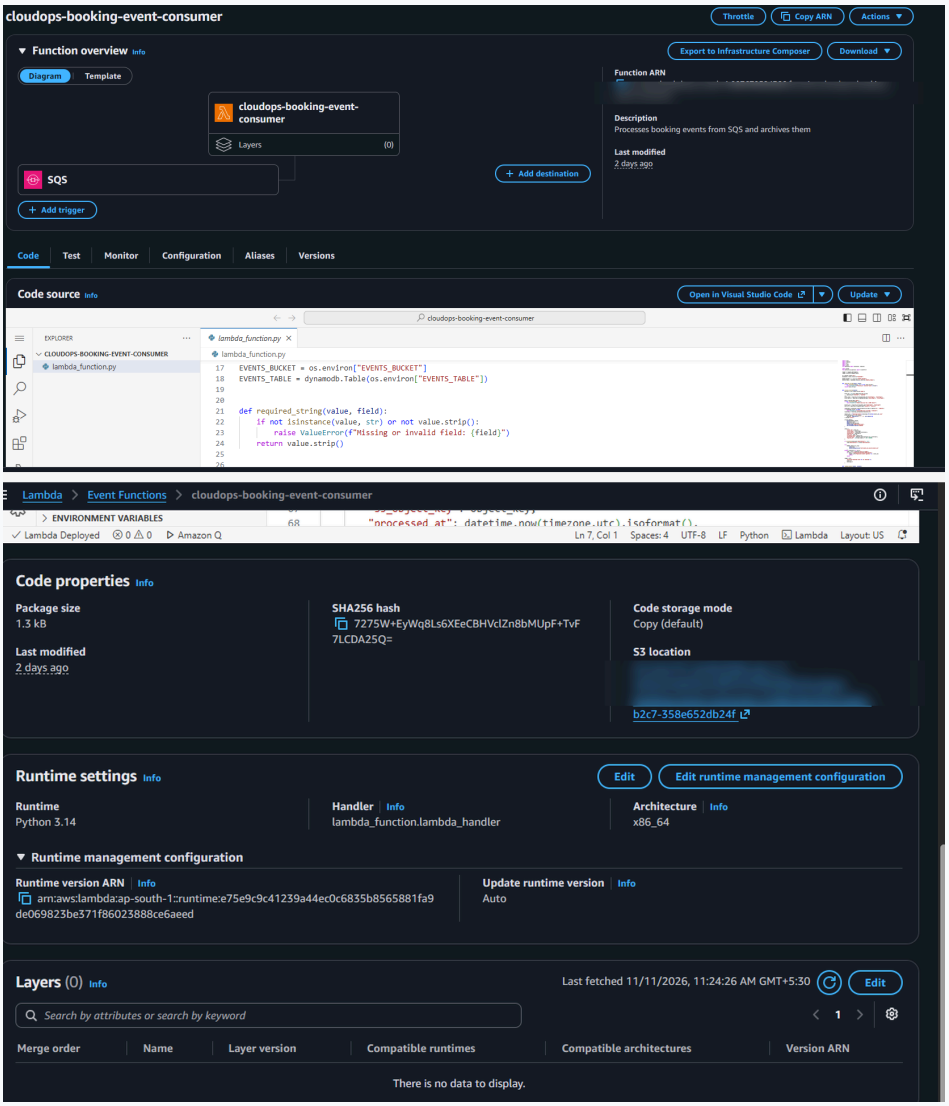
```
cd ~/projects/CloudOps-Enterprise-Platform/serverless/lambda/booking-event-consumer
python3 -m py_compile lambda_function.py
python3 -c 'import zipfile; z=zipfile.ZipFile("function.zip", "w", zipfile.ZIP_DEFLATED); z.write("lambda_function.py");
z.close()'
```

The generated function.zip is ignored by Git. Terraform references the ZIP file from the Lambda resource. The package was uploaded by Terraform, then the SQS event-source mapping was enabled.

4.3 Important deployment correction

The first Lambda configuration requested reserved concurrency of 2. AWS rejected that setting because the account-level unreserved concurrency minimum could not be maintained. The unsupported reserved_concurrent_executions setting was removed. Event-source mapping maximum concurrency remains set to 2. The function and mapping were then verified as active/enabled.

SCREENSHOT SLOT 06 | LAMBDA FUNCTION AND EVENT SOURCE MAPPING



Captured 11 November 2026. Combined AWS Lambda function overview with SQS trigger and the Lambda runtime settings showing Python 3.14.

9. Step 5: Start PostgreSQL and the Microservices

The three service databases were run as separate PostgreSQL 16 containers. The application services and gateway were run with the root compose.yaml. Database containers use named volumes; do not remove these volumes during routine cleanup.

5.1 Prepare environment files

Create local ignored .env and .env.docker files from the service .env.example files. Use strong development-only passwords and JWT secrets. Never commit real .env files or show them in screenshots.

5.2 Start the database containers

```
cd ~/projects/CloudOps-Enterprise-Platform/application/services/auth-service
docker compose up -d

cd ../booking-service
docker compose up -d

cd ../notification-service
docker compose up -d
```

5.3 Connect databases to the shared application network

```
docker network inspect cloudops-platform >/dev/null 2>&1 || docker network create cloudops-platform

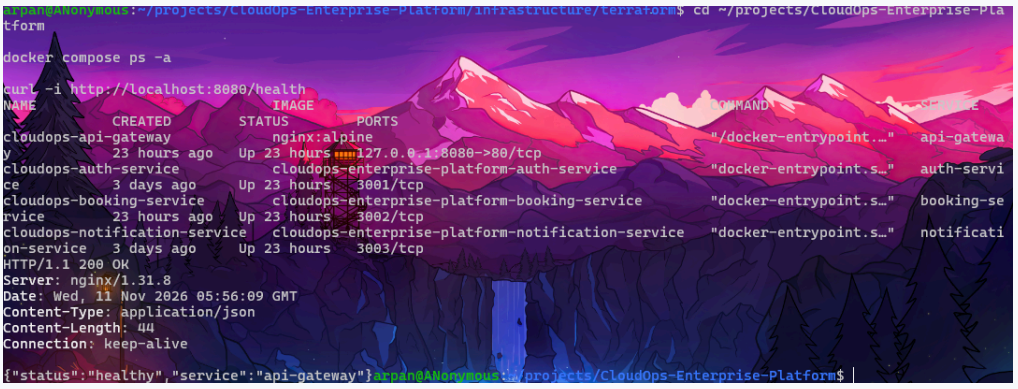
docker network connect cloudops-platform cloudops-auth-db
docker network connect cloudops-platform cloudops-booking-db
docker network connect cloudops-platform cloudops-notification-db
```

5.4 Build and start the application stack

```
cd ~/projects/CloudOps-Enterprise-Platform
docker compose up -d --build
docker compose ps -a
curl -i http://localhost:8080/health
```

The gateway returned HTTP 200 with JSON status healthy. The service containers expose ports 3001, 3002 and 3003 inside the Docker network. Only the gateway port 8080 is published to the local host.

SCREENSHOT SLOT 07 | DOCKER CONTAINERS AND GATEWAY HEALTH



Captured 11 November 2026. Auth, Booking, Notification and Nginx containers running; gateway health endpoint returned HTTP 200.

10. Step 6: Implement Booking API and Transactional Outbox

Auth Service provides registration and login. It hashes passwords and returns a JWT access token. Booking Service protects booking routes and validates the request body before writing to PostgreSQL.

6.1 Apply the Booking Service schema

```
cd ~/projects/CloudOps-Enterprise-Platform/application/services/booking-service
npm run migrate
```

The migration created the booking_event_outbox table. It stores event_id, event_type, payload, status, attempts, available_at, locked_until, last_error, created_at and published_at.

6.2 Booking and event transaction

The createBooking controller starts a database transaction, inserts the booking, generates a booking.created event with a UUID, UTC occurrence time, booking ID, user ID and destination, inserts the event into the outbox, and commits both writes together. On error, the transaction is rolled back.

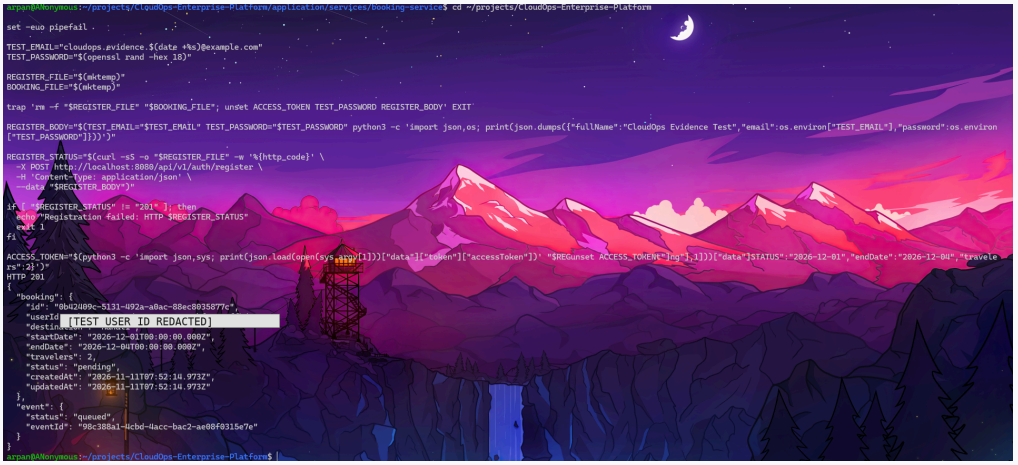
6.3 Example booking request

```
POST http://localhost:8080/api/v1/bookings/
Authorization: Bearer <ACCESS_TOKEN>
Content-Type: application/json

{
  "destination": "Manali",
  "startDate": "2026-12-01",
  "endDate": "2026-12-04",
  "travelers": 2
}
```

The tested synthetic booking returned HTTP 201 and event status queued. The access token is intentionally omitted from this document.

SCREENSHOT SLOT 08 | BOOKING API RESPONSE



Captured 11 November 2026. Booking API returned HTTP 201 and event status queued. The publisher token for this specific event later expired; the separate successful event is documented in Screenshot 09. Test user ID redacted.

11. Step 7: Configure the SQS Publisher Role and Worker

A separate publisher role was created to avoid embedding broad AWS credentials in the Booking Service container. The role policy grants sqs:SendMessage on the booking queue. The local Terraform CLI identity was granted permission to assume that role for the demonstration.

7.1 Configure the queue URL and region

```
cd ~/projects/CloudOps-Enterprise-Platform/infrastructure/terraform
terraform output -raw booking_events_queue_url

cd ~/projects/CloudOps-Enterprise-Platform/application/services/booking-service
# Set AWS_REGION=ap-south-1 and BOOKING_EVENTS_QUEUE_URL in the ignored local .env file.
```

7.2 Assume the scoped role with temporary credentials

```
# Stop any worker using expired credentials with Ctrl+C before this block.
# Clear expired temporary credentials, then confirm the base AWS CLI identity.
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN
AWS_SECURITY_TOKEN
aws sts get-caller-identity --query Arn --output text

# Assume the scoped publisher role and receive fresh temporary credentials.
ROLE_ARN="arn:aws:iam::[REDACTED]:role/cloudops-booking-event-publisher-role"
CREDS="$(aws sts assume-role \
  --role-arn "$ROLE_ARN" \
  --role-session-name cloudops-submission-worker \
  --duration-seconds 3600 \
  --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]' \
  --output text --no-cli-pager)" || exit 1

read -r AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN <<< "$CREDS"
unset CREDS
export AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN

# Confirm the assumed role without printing credential values.
aws sts get-caller-identity --query Arn --output text

# Start the worker in this same terminal so it inherits the fresh credentials.
cd ~/projects/CloudOps-Enterprise-Platform/application/services/booking-service
node src/workers/outbox.worker.js
```

The output showed an assumed-role ARN for cloudops-booking-event-publisher-role. Do not print or share the temporary key values.

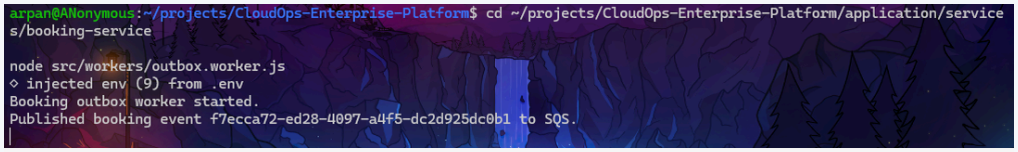
7.3 Start the worker

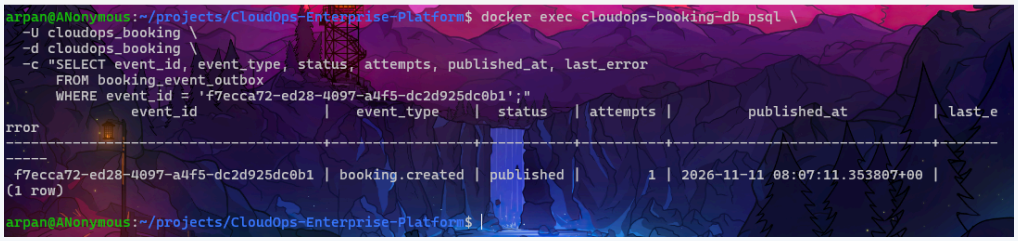
```
node src/workers/outbox.worker.js
```

The worker polls pending rows, claims batches using FOR UPDATE SKIP LOCKED, and publishes each event to SQS. Successful records are marked published; failed sends are retried with bounded delay. The worker uses the AWS SDK credential chain, so its environment must contain valid, unexpired temporary STS credentials. If credentials expire, stop the worker and repeat Step 7.2 before restarting it. Capture the successful worker output as Screenshot 09 in Section 15.

Credential-expiration evidence from the follow-up run: event 98c388a1-4cbd-4acc-bac2-ae08f0315e7e exhausted 10 worker attempts and was marked failed because the STS security token had expired. After refreshing the credentials, a later worker session logged successful publication of a different event, f7ecca72-ed28-4097-a4f5-dc2d925dc0b1. These are different event IDs. The failed 98c... record was not automatically recovered by publishing f7...; verify each event using its own exact ID.

SCREENSHOT SLOT 09 | OUTBOX WORKER AND POSTGRESQL PUBLICATION





Captured 11 November 2026. Worker logged successful SQS publication for event f7ecca72-ed28-4097-a4f5-dc2d925dc0b1; PostgreSQL showed the same event as published after one attempt.

12. Step 8: Verify the End-to-End Booking Event

A synthetic test user was registered through the API. A booking for Manali was submitted through the Nginx gateway. The API returned HTTP 201 and the event ID was recorded in the response.

Follow-up evidence run: a later API booking returned HTTP 201 and was assigned event ID 98c388a1-4cbd-4acc-bac2-ae08f0315e7e. That event's publisher attempt failed because the temporary STS credentials expired. After clearing the expired environment variables and assuming the publisher role again, the worker published a separate new event with ID f7ecca72-ed28-4097-a4f5-dc2d925dc0b1. The SQL evidence for the earlier event must not be used to claim that the later event was published.

12.1 Verify the PostgreSQL outbox

```
docker exec cloudops-booking-db psql \
  -U cloudops_booking \
  -d cloudops_booking \
  -c "SELECT event_id, event_type, status, attempts, published_at, last_error
      FROM booking_event_outbox
      WHERE event_id = 'f7ecca72-ed28-4097-a4f5-dc2d925dc0b1';"
```

The initial API-created event (4377e2b7-1e97-4575-bfee-240d71f627b4) was fully verified earlier: PostgreSQL marked it published and matching data was stored in DynamoDB and S3. The follow-up event f7ecca72-ed28-4097-a4f5-dc2d925dc0b1 was separately verified in PostgreSQL as published after one attempt, with a populated published_at timestamp and no last_error.

12.2 Verify DynamoDB metadata

```
aws dynamodb get-item \
  --table-name cloudops-booking-events \
  --key '{"event_id":{"S":"f7ecca72-ed28-4097-a4f5-dc2d925dc0b1"}}' \
  --consistent-read \
  --region ap-south-1 \
  --no-cli-pager
```

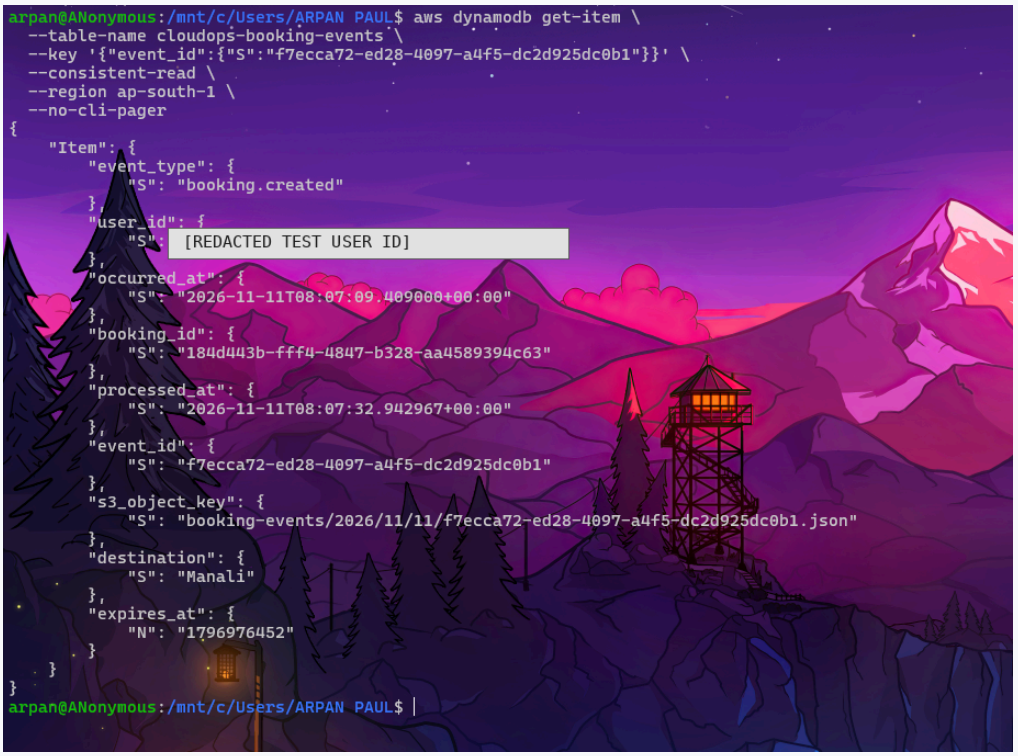
For follow-up event f7ecca72-ed28-4097-a4f5-dc2d925dc0b1, a consistent DynamoDB read returned event_type booking.created, booking_id 184d443b-ffff-4847-b328-aa4589394c63, destination Manali, occurred_at 2026-11-11T08:07:09.409000+00:00, processed_at 2026-11-11T08:07:32.942967+00:00, and the S3 object key booking-events/2026/11/11/f7ecca72-ed28-4097-a4f5-dc2d925dc0b1.json. The test user ID is redacted in the screenshot.

12.3 Verify the S3 event object

```
aws s3api head-object \
  --bucket cloudops-booking-events-[REDACTED]-ap-south-1 \
  --key booking-events/2026/11/11/f7ecca72-ed28-4097-a4f5-dc2d925dc0b1.json \
  --region ap-south-1 \
  --no-cli-pager
```

For follow-up event f7ecca72-ed28-4097-a4f5-dc2d925dc0b1, S3 HeadObject returned content type application/json, content length 251 bytes, ServerSideEncryption AES256, and lifecycle expiration on 12 December 2026.

SCREENSHOT SLOT 10 | DYNAMODB EVENT RECORD



Captured 11 November 2026. DynamoDB returned the follow-up event record with matching event ID, booking ID, destination and S3 key. Test user ID redacted.

SCREENSHOT SLOT 11 | S3 OBJECT PROPERTIES



Captured 11 November 2026. S3 HeadObject confirms JSON content, 251 bytes, AES256 server-side encryption and lifecycle expiration on 12 December 2026.

13. Step 9: Test Failure Handling and Dead-Letter Queue

A deliberately malformed event with an invalid UUID was sent to the main SQS queue. Lambda rejected it during UUID validation and CloudWatch Logs recorded the ValueError. SQS kept the message available for retry; it was later observed in the DLQ.

13.1 Send a controlled invalid event

```
cd ~/projects/CloudOps-Enterprise-Platform/infrastructure/terraform
MAIN_QUEUE_URL="$(terraform output -raw booking_events_queue_url)"
INVALID_EVENT='{"eventId":"not-a-valid-uuid-dlq-
test","eventType":"booking.created","occurredAt":"2026-11-10T00:00:00Z","data":{"bookingId":"invalid-
booking","userId":"invalid-user"}}'
aws sqs send-message --queue-url "$MAIN_QUEUE_URL" \
  --message-body "$INVALID_EVENT" --region ap-south-1 --no-cli-pager
```

13.2 Review Lambda logs and queue counts

```
aws logs tail /aws/lambda/cloudops-booking-event-consumer --since 15m --region ap-south-1 --format short

aws sqs get-queue-attributes --queue-url "$MAIN_QUEUE_URL"   --attribute-names ApproximateNumberOfMessages
ApproximateNumberOfMessagesNotVisible   --region ap-south-1 --output json
unset MAIN_QUEUE_URL INVALID_EVENT
```

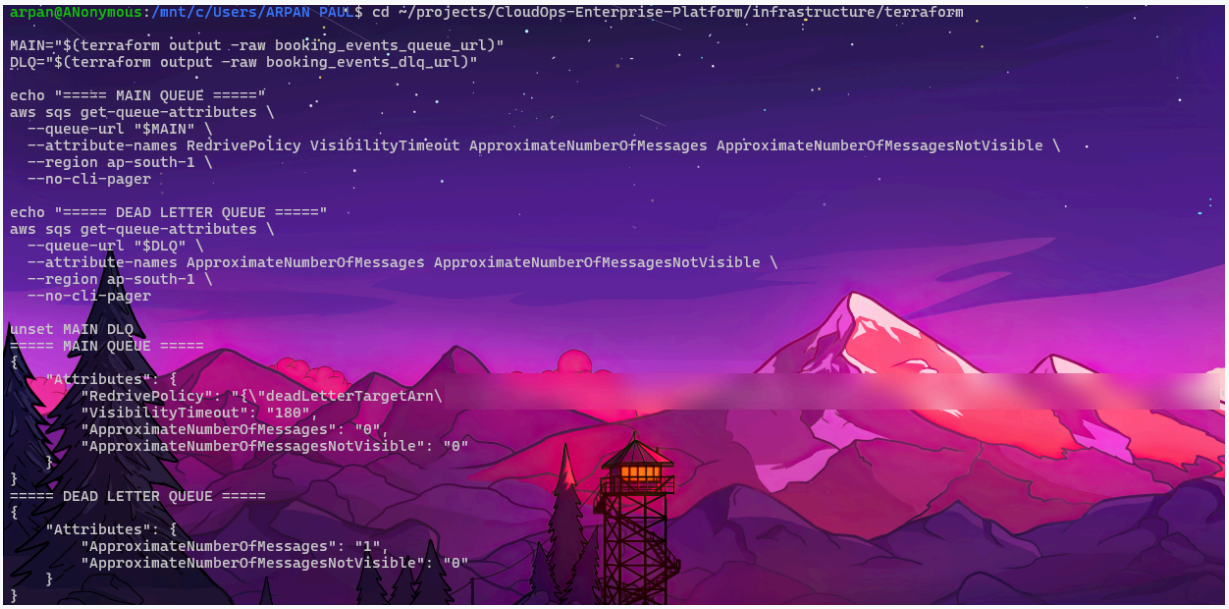
The first DLQ test was run with maxReceiveCount = 3. After the message was confirmed in the DLQ, Terraform was updated to maxReceiveCount = 5. The final applied queue policy was then verified from AWS as maxReceiveCount 5 and VisibilityTimeout 180. The five-receive setting is the final configuration; the earlier failure test itself occurred under the value 3.

13.3 Apply the final redrive threshold

```
cd ~/projects/CloudOps-Enterprise-Platform/infrastructure/terraform
# In serverless-storage-queues.tf set maxReceiveCount = 5
terraform fmt
terraform validate
terraform plan -out=retry-policy.tfplan
terraform apply retry-policy.tfplan
```

The final plan was 0 to add, 1 to change in place and 0 to destroy. Apply succeeded. AWS get-queue-attributes confirmed maxReceiveCount 5 and VisibilityTimeout 180.

SCREENSHOT SLOT 12 | DLQ AND FINAL RETRY POLICY



File: evidence/screenshots/12_dlq_and_final_retry_policy.png | Shows: main queue 0 available / 0 in-flight; DLQ 1 visible / 0 in-flight; visibility timeout 180 seconds. The final maxReceiveCount = 5 policy was separately confirmed by AWS CLI after Terraform apply.

14. Step 10: Configure and Verify GitHub Actions CI

The GitHub Actions workflow runs on push, pull_request and manual workflow_dispatch events. It uses Node.js 22, installs dependencies for the three services, checks JavaScript syntax, builds Docker images, and validates the root Docker Compose configuration.

14.1 CI workflow stages

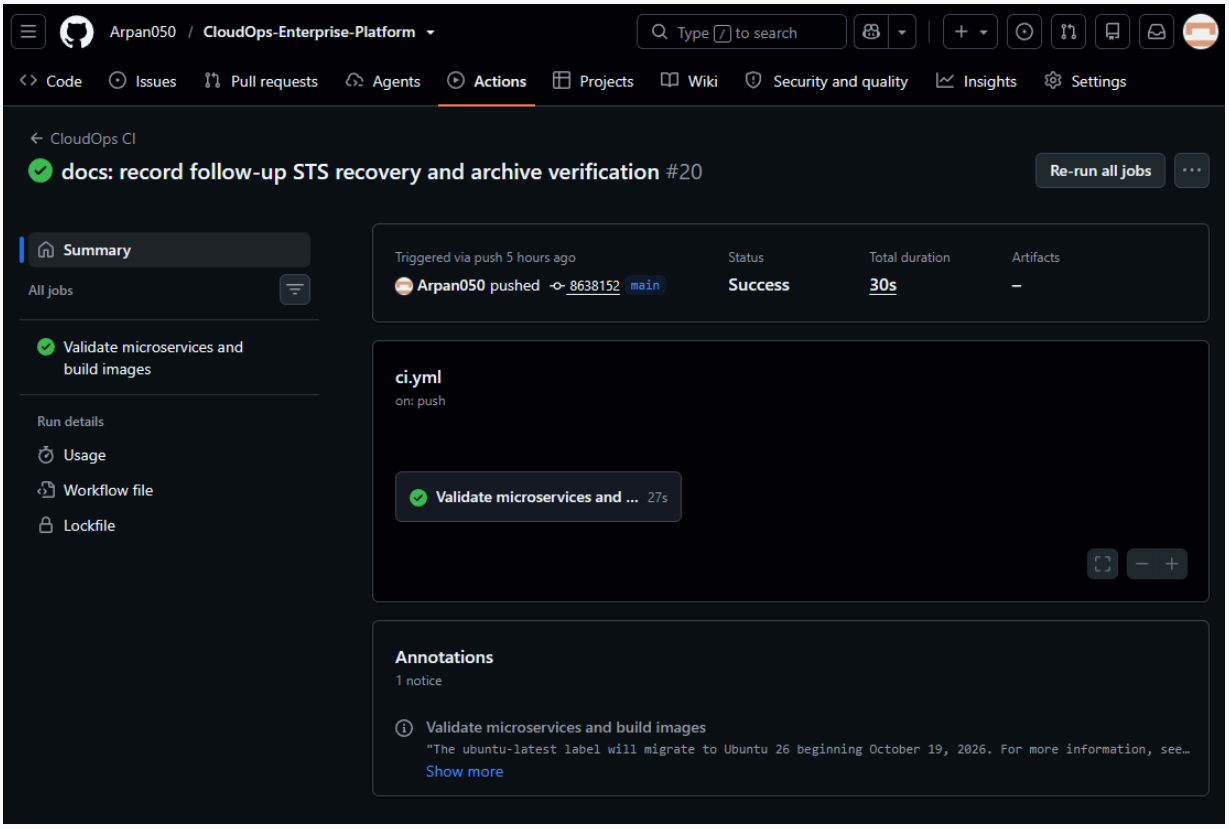
Stage	What the workflow does
Checkout	Checks out the repository.
Node setup	Sets up Node.js 22.
Dependency install and syntax check	Runs npm ci and node --check for JavaScript files in Auth, Booking and Notification services.
Docker builds	Builds one Docker image for each of the three services.
Compose validation	Creates empty ignored .env.docker placeholders and runs docker compose config --quiet.

14.2 Verify the successful run

GitHub Actions run #20 completed successfully for the follow-up STS recovery and archive verification documentation commit. Run #11 also passed for the implementation commit that finalized the SQS retry setting and Booking Service SDK dependency. The workflow is CI only; it does not deploy the application to AWS.

```
git status --short
git log -1 --oneline
git push origin main
```

SCREENSHOT SLOT 13 | GITHUB ACTIONS SUCCESSFUL RUN



File: evidence/screenshots/13_github_actions_successful_run.png | Shows: CloudOps CI run #20 completed successfully, including the microservice validation and Docker build job.

15. Deliverables: Screenshots

All thirteen screenshot slots have now been populated with captures collected during implementation and verification. Each image is placed beside the related implementation step; the checklist below describes the evidence each capture provides.

#	Screenshot	What it should prove
1	Environment versions	Terraform, AWS CLI, Docker, Compose, Node/npm and Git versions.
2	Terraform init and validation	Provider initialization and successful terraform validate.
3	Terraform final plan	A current no-change plan, or the saved historical retry-policy plan if it is still available; do not recreate historic apply output.
4	AWS VPC and subnets	CloudOps VPC CIDR, four subnets, route tables and security groups.
5	SQS and DLQ settings	Visibility timeout 180 seconds, maxReceiveCount 5 and DLQ settings.
6	Lambda function and mapping	Function overview, runtime and Enabled SQS event-source mapping.
7	Docker services and gateway	Application containers running and gateway HTTP 200 health response.
8	Booking API response	HTTP 201, booking ID and event status; JWT hidden.
9	Outbox worker and PostgreSQL	Successful publish log and outbox row status published.
10	DynamoDB event record	Matching event ID, booking ID, destination and S3 key; redact user ID.
11	S3 object properties	JSON archive, content length, AES256 encryption and lifecycle expiration.
12	DLQ and queue health	Actual terminal capture: main queue 0 visible / 0 in-flight; DLQ 1 visible / 0 in-flight; visibility timeout 180 sec. Final redrive threshold maxReceiveCount = 5 was separately verified after Terraform apply.
13	GitHub Actions	GitHub Actions CloudOps CI run #20 completed successfully; green job shows microservice validation and image-build workflow.

16. Troubleshooting

The following table records issues encountered during this implementation and the actions used to resolve them.

Problem encountered	Resolution / lesson
Docker command not found in WSL 2	Enabled Docker Desktop WSL 2 engine and the integration for the Ubuntu distribution; reopened the WSL terminal.
Nginx gateway exited with code 127	Docker reported a bind mount type mismatch for nginx.conf. Verified the host path was a regular file, removed and recreated only the gateway container. The health endpoint returned HTTP 200.
AWS SDK npm install failed with SSL cipher error	Verified the npm registry was reachable, cleaned the npm cache and retried installation. The SQS SDK installed successfully.
Lambda deployment rejected reserved concurrency = 2	Removed reserved_concurrent_executions. Kept maximum event-source concurrency = 2 on the SQS mapping. Untainted/recovered the function state and applied the event-source mapping.
DynamoDB item was not visible immediately after sending the test event	Allowed asynchronous SQS/Lambda processing to complete, checked CloudWatch logs and repeated a consistent read.
Invalid message remained in-flight	Reviewed the 180-second visibility timeout and SQS retry behavior. Waited for Lambda retries; verified that the message later appeared in the DLQ.
Terraform retry setting was below the desired production threshold	Updated maxReceiveCount from 3 to 5, reviewed a one-resource in-place plan and applied it. AWS returned the final value 5.
Terminal paste/output became interleaved or truncated	Split long commands into shorter blocks and used separate terminals for the worker and verification commands.
Outbox publisher STS token expired	The worker logged “The security token included in the request is expired”. The event 98c388a1-4cbd-4acc-bac2-ae08f0315e7e reached failed after 10 attempts. Stop the worker, unset AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN and AWS_SECURITY_TOKEN, verify the base AWS CLI identity, assume the publisher role again, export the fresh credentials, then restart the worker. A separate event f7ecca72-ed28-4097-a4f5-dc2d925dc0b1 was then published. Do not confuse the two event IDs.

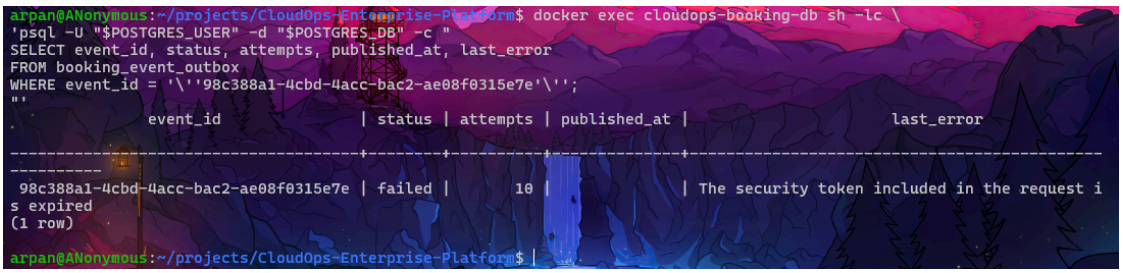


Figure: PostgreSQL shows the earlier event marked failed after 10 attempts due to an expired STS token.

17. Clean Up and Resource Management

The resources are retained for the project demonstration. Cleanup must be performed carefully because the AWS Terraform state contains the CloudOps VPC and event-processing resources, while local PostgreSQL data is stored in Docker volumes.

Local process shutdown

```
# In the terminal running the outbox worker, press Ctrl+C.

cd ~/projects/CloudOps-Enterprise-Platform
docker compose stop

docker stop cloudops-auth-db cloudops-booking-db cloudops-notification-db
```

These commands stop the local application and database containers. They do not remove named database volumes. Do not run docker compose down -v for this project data.

AWS cleanup

Do not run terraform destroy while the infrastructure is still required for submission or demonstration. Review the Terraform plan and AWS resource inventory before removing anything. Usage-based services may incur charges while retained. Verify actual spend in AWS Billing; a budget alert is not necessarily a hard spending cap.

18. Conclusion

The project demonstrates a working cloud and DevOps integration path: Terraform-managed AWS networking, Dockerized microservices, a transactional outbox, scoped IAM role assumption, SQS-based event delivery, Lambda processing, encrypted S3 archives, DynamoDB metadata, CloudWatch logging and dead-letter handling. A real API-created booking was followed through PostgreSQL, SQS, Lambda, DynamoDB and S3. A malformed event was rejected and later routed to the DLQ. The final Terraform retry-policy change was applied without creating or destroying resources, and the latest GitHub Actions workflow passed.

The project remains a partial implementation of the full eight-workstream brief. The local services were not deployed into the AWS VPC; RDS, continuous deployment, complete monitoring dashboards, Big Data/AI and no-code integration were not implemented. These limitations are recorded instead of being claimed as completed.

19. Learning Outcomes

- Writing and organizing reusable AWS infrastructure using Terraform and HCL.
- Understanding Terraform init, fmt, validate, plan, apply and state-based change management.
- Designing VPC CIDR ranges, subnets, route tables, Internet Gateway paths and tiered security groups.
- Building containerized Node.js services and routing requests through Nginx.
- Using PostgreSQL transactions and the transactional outbox pattern to avoid losing events after booking commits.
- Publishing to SQS with short-lived STS credentials and a resource-scoped IAM role.
- Processing SQS messages with Lambda, including partial batch failures, idempotency and DLQ redrive behavior.
- Storing raw event archives in encrypted S3 and searchable metadata in DynamoDB with TTL.
- Using CloudWatch Logs to investigate successful processing and validation failures.
- Creating a GitHub Actions CI workflow that installs dependencies, checks source syntax, builds images and validates Compose.
- Distinguishing a deployed/verified capability from a planned or documentation-only module.

End of implementation work document.